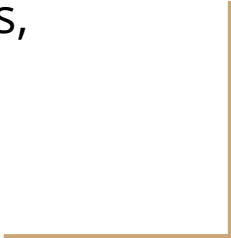




Linux Containers, 101

alvaro barcellos,
SGB, 09/2026,
version 1.0





How big is four bytes (2^{32})?

$$1 \text{ Gigabyte (Gb)} == 2^{10} * 2^{20} == 1.048.576_{10} > 10^6$$

$$1 \text{ Terabyte (Tb)} == 2^{20} * 2^{20} == 1.099.511.627.776_{10} > 10^{12}$$

$$1 \text{ Petabyte (Pb)} == 2^{30} * 2^{20} == 1.125.899.906.842.624_{10} > 10^{15}$$

$$1 \text{ Exabyte (Eb)} == 2^{40} * 2^{20} == 1.152.921.504.606.846.976_{10} > 10^{18}$$

$$32 \text{ bits} == 2^{12} * 2^{20} == 2^{32} == (4096 * 1 \text{ Gb}) == 4.294.967.296_{10} \text{ 4 bytes}$$

$$64 \text{ bits} == 2^{24} * 2^{40} == 2^{64} == 18.446.744.073.709.551.616_{10} \text{ 8 bytes}$$

$$\text{segundos em 1 ano: } 365 * 24 * 60 * 60 == 31.536.000_{10}$$

by 1 tic-tac per second ~ 584 billion years ~ 40 times age of universe

UUID

A UUID (Universal Unique Identifier) is a set of 16 bytes, represented by a 36-character string containing 32 hexadecimal digits separated by hyphens in an 8-4-4-4-12 pattern, generated by common and public well-known algorithms.

RFC: <https://datatracker.ietf.org/doc/html/rfc4122>

eg: 05091d90-aa15-11f1-863d-00155df45fca

“You would need to generate 1 billion UUIDs per second for 100 years to reach a 50% chance of a single collision” (128 bits)

Hash, SHA256

SHA-256 (Secure Hash Algorithm) is a set of 32 bytes, represented by 64 hexadecimal characters, generated by a one-way cryptographic hash function that takes an input of any size and deterministically maps it to a unique hash.

RFC: <https://datatracker.ietf.org/doc/html/rfc6234>

01ba4719c80b6fe911b091a7c05124b64eece964e09c058ef8f9805daca546b

“To put the magnitude into perspective, the probability of one billion arbitrary files colliding is about 4.3×10^{-60} ” (256 bits)

(also could be 140, 512, 1024, 2048, 3072, 4096 bits)

Digital signature

A pair of large numbers (32 to 64 bytes) used as asymmetric keys in digital systems for transport secure layers (TSL) and secure socket layers (SSL).

Public Key	Private key
Used to validate signature	Used for unique signing
Used to encrypt a message	Used to decrypt a message
Publicly shared by a reliable agency	Only the owner has the key
Cannot guess private key	Cannot guess public key

Bob and Alice shares a secret

1. Alice sends a `KeyUpdate` to Bob
2. Bob take a UUID and encrypt it with the public key of Alice
3. Bob sends the encrypted UUID to Alice
4. Alice receive the encrypted UUID and decrypt with her private key
5. Alice uses UUID to encrypt data and send to Bob
6. Bob uses UUID to encrypt data and send to Alice
7. Repeat in a random time interval

Time based ~ On Time Password

TOTP is a temporary numeric code used as challenge to authentication. Usually with 6 digits and valid for 30 seconds.

It is based on RFC-6238, which describes a generator based in a shared secret and an universal time from unix epoch, 00:00 01-01-1970. The secret is encoded as RFC-4648, Base32, with padding omitted.

The generator is SHA-1 (160 bits) or SHA-256 (256 bits).

The Authenticators uses a QRcode with `otpauth://totp/` to synchronize the challenge parameters.

Usually the secret is a random UUID shared by server to client.



UTF-8

It stands for Universal Character Set Transformation Format - 8-bit, used in all Internet browsers. RFC 3629.

Uses one byte for 1960's ASCII english characters and, for special characters, accents, emojis, and non-Latin alphabets, (like Chinese, Arabic etc) uses 2 to 4 bytes.

Resolve the critical external "code page" identification issue that generates records containing incompatible and difficult-to-understand information.

Ken Thompson and Rob Pike created UTF-8. They designed it in September 1992.

MIME

Multipurpose Internet Mail Extensions types are all standardized in IETF's [RFC 6838](#). Used in web Content-type: type/sub-type

text/plain, text/css, text/html, text/javascript,

application/octet-stream, application/JSON, application/pdf

image/jpeg, image/png, image/svg+xml

audio/mpeg, video/mp4, font/woff, font/otf

model/3mf, model/vrml

more on <https://www.iana.org/protocols>

SGML

Standard Generalized Markup Language (1986), RFC 1874

as tagged language defined by pairs `< something > ... </ something >`

Hyper Text Markup Language (HTML) (1995), RFC 1866, includes properties and attributes for web files, as `key="value"`

Extensible Markup Language (XML) (1998), RFC 3023, RFC 5364, define format extension for representing parameters in resource lists;

CSS, SVG, MathML, XHTML, etc, emerged from XML.

SGML → HTML → XML → JSON → YAML

JSON, JavaScript Object Notation

JSON Schema is a declarative language for annotating and validating JSON documents' structure, constraints, and data types (2002).

It uses tuples $\rightarrow$ "identifier" : "content", and contents could be:

number $\rightarrow$ 0.7654, +7.564e-1,	(IEEE 754 format)
string $\rightarrow$ "this is a string" or null,	(not "null")
boolean $\rightarrow$ false OR true,	(not FALSE or "false")
array $\rightarrow$ ["first", "second", "third"],	(array of strings)
object $\rightarrow$ { "key1":12345, "key2":"12345", "key3":[1, 2, 3, 4, 5] },	

could have arrays of arrays of objects with different identifiers

JSON,

```
{
  "first_name": "George", "last_name": "Washington",
  "address": {
    "street_address": "3200 Mount Vernon Memorial Highway",
    "city": "Mount Vernon", "state": "Virginia", "country": "United States"
  }
  "birthday": "1732-02-22"
}
{
  "comments": [
    "# The braces { } define the structure's order and hierarchy.",
    "# note the end of tuple comma", "# uses backslash to escape \" ",
    "# there is no comments" ]
  }
```

JSON,

<https://www.json.org/json-en.html>

<https://ecma-international.org/publications-and-standards/standards/ecma-404/>

[https://ecma-international.org/wp-content/uploads/ECMA-404_2nd_edition_dece
mber_2017.pdf](https://ecma-international.org/wp-content/uploads/ECMA-404_2nd_edition_dece_mber_2017.pdf)

<https://www.rfc-editor.org/info/rfc8259/>

Defined in a six pages only document.

Any valid JSON file is automatically a valid YAML file.

But there is no comments in a JSON file. (: critical fail :)



YAML, Yaml Ain't Markup Language

A human readable data serialization used for configuration files, data exchange and automation (2006). Simple syntax rules (?).

It uses tuples of key-value and python style indentation with (must be) multiples of 4 spaces (never tabs).

Most basic rules:

- Key-value pairs, separated with space and colon (:)

- List are denoted by a dash (-) and space

- Comments are started by a hash (#) symbol

- Uses anchors (&), references (*) and aliases (<<:)

YAML

```
"first_name" : "George"  
"last_name" : "Washington"  
"address" :      # last known address  
  "street_address" : "3200 Mount Vernon Memorial Highway"  
  "city" : "Mount Vernon"  
  "state" : "Virginia"  
  "country": "United States"  
"birthday" : "1732-02-22"  
  
# allow comments anywhere  
# one tuple by line, no comma separator  
# uses backslash to escape \#
```

YAML,

YAML is a trap, it hides complex scopes. (anchors, references, templates etc).

Extensive rules of context, tags, sets, mappings, scalars, sequences.

Multi record and document capable.

<https://yaml.org/>

<https://yaml.org/spec/1.2.2/>



Networks

IPv4, Internet Protocol version 4, is a system used to identify devices on a network. RFC 791, RFC 1122, RFC 1918, RFC 5735. Authority IANA

Uses 4 bytes to represent device address and network masks.

Network masks use bitwise operations (CIDR), to define groups of directly connected devices. Each device have a unique 6 byte MAC address.

Devices on same network could talk directly but to talk with other networks, the packets must be forwarded by network gateways.

[header IPv4 [header TCP/UDP/SCTP/QUIC/ICMP [header APPs [payload]]]]

Use 1420 MTU for best speed

OSI Model Layers Explained Simply

L1 Physical

- Cables & Signals



- Hub, Cables
- Hardware Level

L2 Data Link

- MAC Address



- Switch, VLANs
- Local Network

L3 Network

- IP Address



- Router, IP
- Internet Level

L4 Transport

- Ports & Protocols



- TCP, UDP Ports
- Application Level

Cable → Switch → Router → Port

Physical → MAC → IP → Port

Classes of IP Addresses (IPv4) – Explained Simply

Class	Range	Used By	Example Use Case
	0.0.0.0 – 127.255.255.255	Large networks	Internet providers
	128.0.0.0 – 191.255.255.255	Medium-sized networks	Colleges, organizations
	192.0.0.0 – 223.255.25.5.255	Small networks	Homes, small businesses
	224.0.0.0 – 239.255.25.5.255	Multicasting	Streaming to multiple devices
	240.0.0.0 – 255.255.25.5.255	Research & Reserved	Experimental use

Networks, non internet routable

Address only valid in an internal networks.

Address/Mask	Present Use
10.0.0.0/8	private use (16.777.216)
127.0.0.0/8	loopback
172.16.0.0/12	private use (262.144)
192.168.0.0/16	private use (65.536)



Memory Maps

Memory mapping is a technique used in computing systems to manage and allocate memory resources efficiently.

Maps pages of real physical memory into virtual logical segments and keep track of allocation. Uses 32 or 64 bits spaces.

A special circuit MMU, optimizes memory caches and access.

The operational system relies in policies to management memory, usually “Buddy Blocks”, using pages, multiples of 4kb, and trying to keep low fragmentation allocation.

These techniques are also used for disk block allocation.

Maple Trees

The Maple Tree is a recent (2022) B-Tree data type optimized for storing non-overlapping ranges and CPU cache line size (64 bytes).

Optimized for storing sequential ranges and safe RCU by spinlock.

Each node have 256 bytes and 256 bytes aligned, usually with 32 slots of 8 bytes. An 4096 block holds 16 nodes.

A slot of 64-bits could be a pivot, with hash value, or a pointer, to blocks.

One slot always points to parent node, using 7 bytes, and have one byte of metadata, with node type and slots sizes.

Maple tree, example

parent	pointer16
hash1	pointer1
hash2	pointer2
hash3	pointer3
....	
hash13	pointer13
hash14	pointer14
hash15	pointer15

hash1 < hash2 ... < hash15
as U_LONG for comparison.

pointer16 to node with
hashes greater than hash16

pointerN to node with hashes
lesser than hashN

parent includes metadata
about node format and type

allocation node, blocks

parent-s	next
block1	block2
block3	block4
block5	block6
....	
block25	block26
block27	block28
block29	block30

A pivot is enough ?

One byte could represent 256 values ~ about $\sim 26 * 10$ ($\sim 98/100$)

A pivot is $256^8 \sim$ about $\sim (26 * 10)^8 \sim 26^8 * 10^8$

Like a text with 98.000.000 words of 8 characters or
about 100 books with 450 pages with 32 lines by 68 characters

A pivot could use hash of XX64 family, fastest non-crypto ~ 40 Gb/s

(XX64 have 50% chance of a collision for about 5 billion items :)



Unix Philosophy

Write programs that do one thing and do it well.

Write programs to work together.

Write programs to handle text streams.

(because that is a universal interface)

more in <http://www.catb.org/esr/writings/taoup/html/ch01s06.html>

and Linus says "Do not break userspace"

Everything is a stream (of bytes)

define, stats,
create, delete,
open, close,
void, seek,
lock, pause,
write, read,
fork, exit



Operating System, basics

Kernel (Scheduling and Multitask)

CPUs+GPUs+MMUs (and Balance ISAs)

Devices (and Modules)

Memory (and Mapping)

Filesystem (and storage Input and Output)

Process (and Libraries)

User (permissions, profiles and limits)

Files

Files are used to store information in blocks of encoded bytes on devices. These blocks are grouped by the file system and have metadata recorded in numerical index nodes (inodes), such as:

- inode number, type (file, pipe, socket, link, device, directory)

- dates (creation, access, modification)

- owner (user and group)

- permissions (execute, write, read, specials)

- links, blocks, or devices (major, minor)

- size, block size, pointer (or list of blocks)

directories only
stores a list of
tuples (pairs) of:
name and inode

Files, use of inodes and blocks

each inode (~256 bytes)			block 0573, crc32		block 6756, crc32		block 5699, crc32
Inodes	metadata*	block	.	0975	.	5698	
0975	directory	0573	..		..	0975	
1001	pipe	0	A	1001	F	6071	
3728	file	5699	B	3728	G	3728	
5698	directory	6756	C	5699	H	6780	

* more info omitted

Directory tree

Disk partitions are mounted onto directories.
Must to have a file system as:

memory: procfs, sysfs, devfs, tmpfs

directory: fusefs, overlayfs

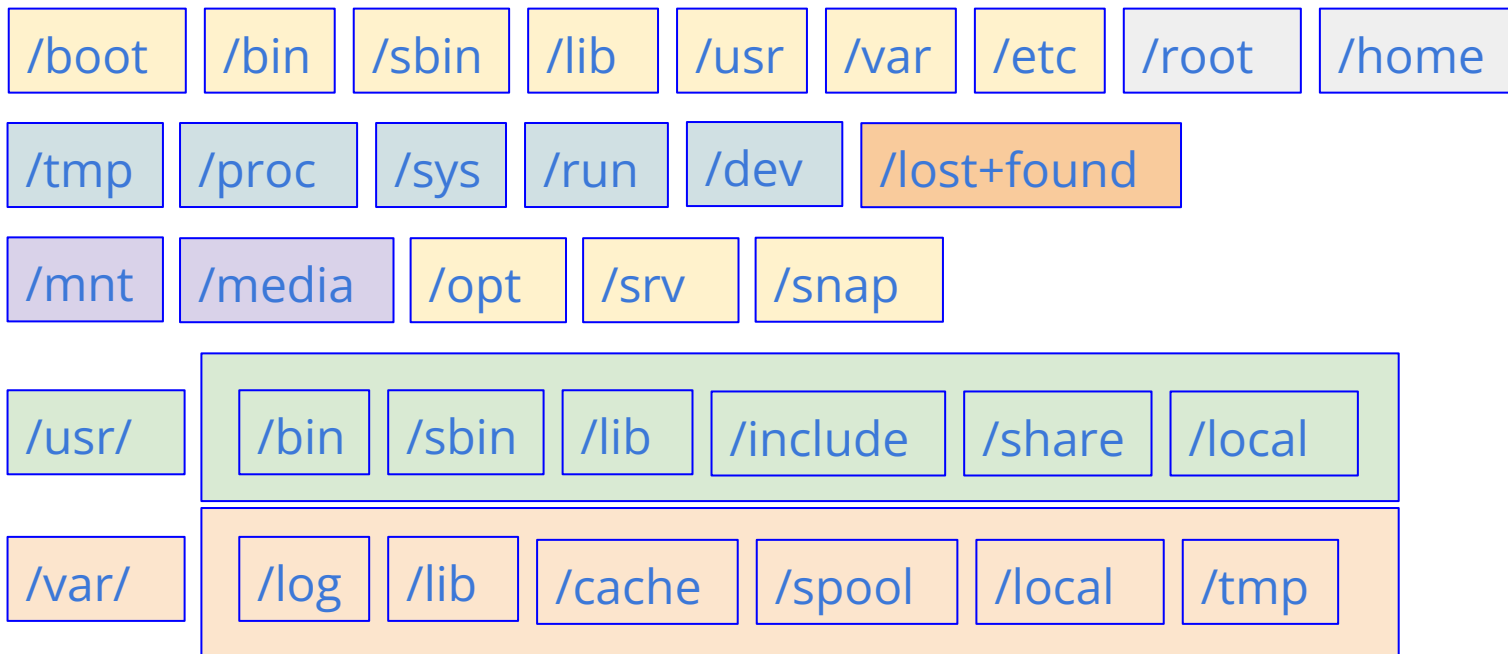
local disks: ntfs, cifs, ext4fs, btrfs, zfs

network: sshfs, zfs, xfs, glusterfs, cephfs, seaweedfs

superblocks, checksums, hashes, bind, journal, snapshot, self-healing, RAID,
CoW, LVM, VFS, LFS, FSCK, etc

Linux directory tree

https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html



Directory tree

Use slash (/) to concatenate directories as the path to a file

the first slash (/) is the root directory

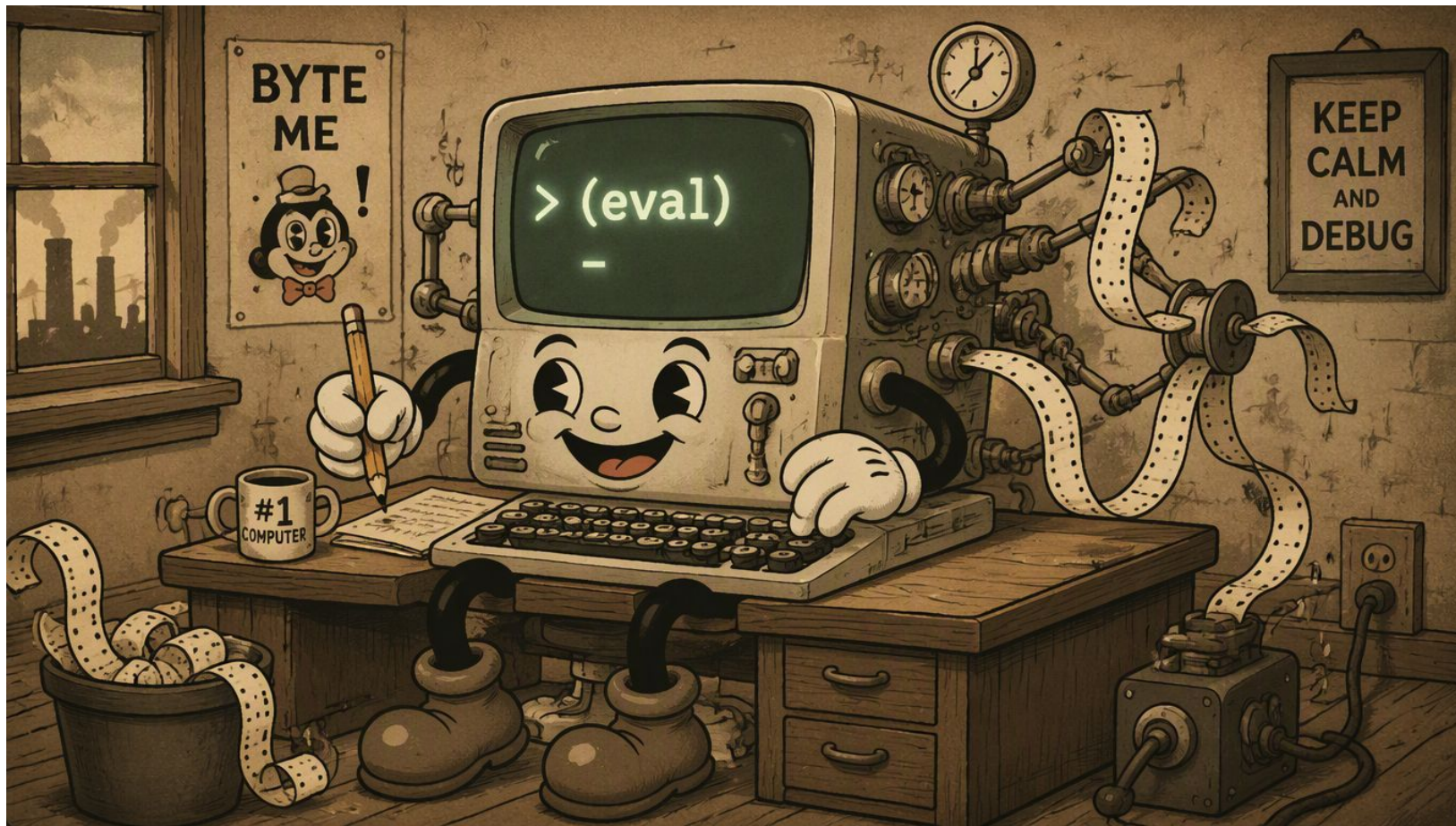
the one dot (.) directory refers to current directory

the two dots (..) directory refers to parent directory

the tilde (~) directory refers to user home directory

in names, use ? to replace one character and * to replace many characters

in names, use single tick (') to encapsulate names with spaces, 'like in this '



Shells

A shell is an interactive command interpreter language.

It have variables, decides and loops.

bash, bourne-again-shell, is the most used shell in linux.

A shell command line is a sequence of arguments separated by spaces and terminated by a semicolon (;) or new-line.

Use quotation marks (" ") ou single tick (' ') to group words.

define environment variables as: NAME="contents"

execute commands as: for file in `ls \*.dot` ; do echo "\$file" ; done

Shells, bash

A process executed by shell returns a numeric error code

0 is no error; 1 is a generic fail; 2 is a command line error;

Others numbers depend on the application. Use `$?` for access last code.

processes can be chained by test the error code as:

`prg1 && prg2 && prg3 && prg4` stops at first non zero returned

`prg1 || prg2 || prg3 || prg4` stops at first zero returned

Shells, bash

It provides a standard redirection of file descriptors:

input is <0 ; output is 1> ; error is 2> ; both 1 and 2 is &>

(0 and 1 could be omitted, others are 2, 3, 4 ...)

process executed by shells, read and write files as:

```
prgA < file1 > file2 ; prgB < file2 > file3 ; prgC < file3 > file4
```

or using pipes as:

```
prgA < file1 | prgB | prgC > file2
```

the > creates a file or truncate contents a file, use >> append to a file

Bash, aliases

`alias name=value`

`alias name='command'`

`alias name='command arg1 arg2'`

`alias name='/path/to/script'`

`alias name='/path/to/script arg1 arg2'`

`alias name='command && command2 && command3'`

`alias name='command || command2 || command3'`

`unalias alias-name`

Bash, aliases

Safer defaults

alias cp='cp -i'

alias mv='mv -i'

alias rm='rm -i'

Network

alias myip='curl -s ifconfig.me'

alias myip6='curl -s ifconfig.me/ip6'

alias ports='ss -tulnp'

System

alias meminfo='free -h'

alias cpuinfo='lscpu'

alias diskinfo='df -h'

alias topmem='ps aux --sort=-%mem \
| head -15'

alias topcpu='ps aux --sort=-%cpu \
| head -15'

Bash, set | less

A variável de ambiente PATH guarda os diretórios onde o shell vai buscar os comandos, a ordem de busca é importante.

```
PATH=/home/user/.local/bin:/usr/local/bin:/usr/local/sbin:/usr/bin:/usr/sbin:/bin:/sbin:
```

```
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:expand_aliases:extglob:extquote:force_fignore:globasciiranges:globskipdots:histappend:interactive_comments:patsub_replacement:progcomp:promptvars:sourcepath
```

```
PS1='${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\$ '
```



Pipes

Pipes are memory buffers for write into and read from one will write and one will read (First In, First Out)

Anonymous pipes are temporary, it connect processes

One | Two | ... | Six | ... | Ten

Named pipes (FIFOs) are permanent in filesystem

PgrOne > Named_Pipe ; PrgTwo < Name_Pipe ;

or use a “publish-subscribe-model” as MQTT for many to many

Sockets

a Socket is like a pair of Pipes.

- it **Bind** to an address (IP V4 or V6) and a port (1-65535)

- it **Listen** for a stream (protocol [http, ssh, dns, nfs, ntc ...])

a server **Accept** a connection from a client **Connect**, and uses:

- Select** (from a list *) or **Poll** (a pointer to a list *)

- Then **Fork** (copy old) or **Spawn** (creates new)

- (* list of sockets connected)

**“In Unix, everything is a file.
If it isn't a file, it's a process ”**

Libraries and Processes

A process is a program executing (running, waiting or blocked).

The code of programs can be compiled as static or shared.

As static, the process have all functions fixed inside it.

As shared, the process uses functions from system libraries.

The libraries are shared between many processes.

It can be upgraded or patched anytime and reduce the use of memory space.

Process

Every process have PID and PPID. ($1 < \text{PID//PPID} < 4.194.304, 2^{22}$)
Any process is child of a parent process. Any process accepts signals.

It could be in foreground (interactive) or in background (no-interactive)

- & next process go to background
- ^z suspends a process to background
- bg running a process in background
- fg running a process in foreground
- jobs lists process in background

states: running or sleep or wait or stop or orphan or zombie



Daemons

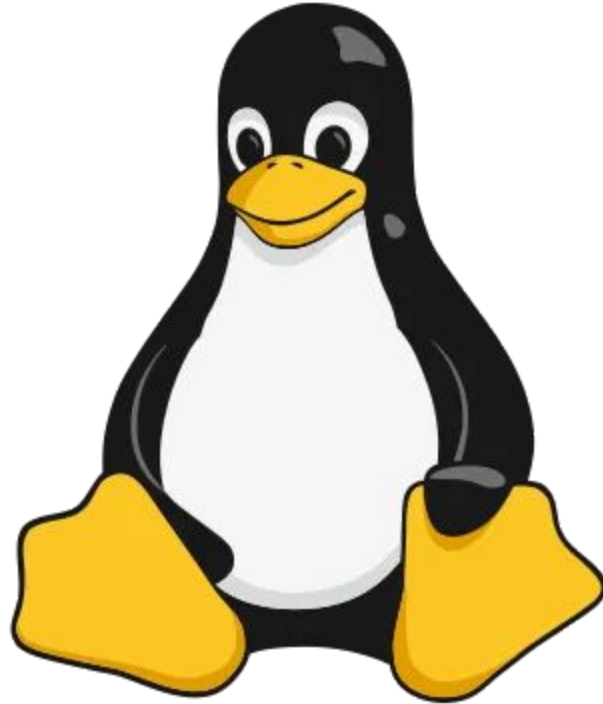
Processes that runs continuously in background without intervention or interaction, are called “daemons”.

A term was used by MIT, from Maxwell’s experiment as “a being controlling a door between two gas chambers”.

Usually named ending in ‘d’ as systemd, uidd, logind, containerd, dockerd, httpd, sshd, rsyslogd ...

Typically, theagetty daemon handles local logins, and sshd handles remote logins.

Tux



**“Unix commands are short
because teletypes ran at 110 baud.”**

Basic commands,

manuals of commands	man
list related man pages	apropos
name actual directory	pwd
change directory	cd
make directory	mkdir -p
remove directory	rmdir -p

Basic commands,

list directory contents	ls
copy files	cp
rename or move files	mv
remove files or directories	rm
link files	ln
copy and modify a file	dd

Basic commands,

report inode attributes	stat
change access attributes	chmod
change owner attributes	chown
change timestamps	touch
make a named pipe	mkfifo
make a temporary file	mktemp

Basic commands,

output its arguments	echo " texts "
copy to a file and output	tee <u>file</u>
concatenate files	cat
split files by size or lines	split
inverse of cat	tac
count lines, words, characters	wc



Basic commands,

list lines interactively	less
list first lines	head
list last lines	tail
sort lines	sort
list unique lines	uniq
compare two sorted files line by line	comm

Basic commands,

format fields of record lines	column
select fields of record lines	cut
merge lines of files	paste
join files by common field	join
change line contents by sets	tr
change line contents by regex	sed

Basic commands,

line-oriented text editor	ed
edit contents of files - visual	nano
edit contents of files + LISP	emacs
edit contents of files (ex update)	vi
edit contents of files + VimL	vim
edit contents of files + LuaJIT	neovim

Basic commands,

compare lines of files	diff
restores files by diffs	patch
list files with selected attributes	find
select files by contents	grep
make an archive of files	tar
compress/uncompress files	gzip or zstd



Basic commands,

determine file type	file
date and time of days	date
generate a UUID	uuid
generate a hash sha256	sha256sum
encode/decode base32/64	base32 or base64

Basic commands,

report directory usage space	du
report disks usage space	df
report disk mounts	mount
report networks stuff	ip
report files stats	ls
report socket stats	ss
statistics for cpus and disks	iostat

Basic commands,

report system information	uname
report process on host	ps
report system stats	top
modify process priority	nice
run a command, no hangups	nohup
send a signal to processes	kill *

- SIGHUP (1), SIGINT (2), SIGQUIT (3), SIGKILL (9), SIGTERM (15), SIGCONT (18) SIGSTOP(19),

Basic commands,

add a user	useradd
modify a user	usermod
delete a user	userdel
change password	password
execute as other user	su requires user passwd
execute as other user	sudo requires own passwd



Basic commands,

manage packages	apt
content version tracker	git
openPGP encryption tools	gpg
PKI encryption tools X.509	openssl
network connection tester	ping
start a shell in remote host	ssh

Basic commands,

copy files with FTP protocol	sftp
copy files in secure ssh tunnel	scp
copy files from remote host	curl
copy files from remote web	wget
copy files and synchronized	rsync

Basic commands,

manages network traffic	tc
multi tool TCP/UDP/IPC	nc
active network explorer	nmap
passive network explorer	snort
manage network firewall	iptables
manage netfilter firewall	ufw

Basic commands,

build automation by dependencies	make
process AWK scripts	awk
process JSON files	jq
process SQLite databases	sqlite3
exclusive host identification	hostid



Basic commands,

manage system control	systemctl
manage system logs	journalctl
manage user resource limits	ulimit
list kernel messages	dmesg
trace functions of process	strace
print shared object dependencies	ldd

Basic commands,

manage schedule of commands	crontab
terminal multiplex	tmux
terminal multiplex	screen
double panel file manager	mc
fuzzy file list finder	fzf
finish a interactive shell	exit



Perfection is attained,
not when there is nothing more to add,
but when there is nothing more to take away

who is root ?

On Linux the most privileged user is the root, uid 0 and gid 0.

It is default superuser account that holds absolute administrative control over the operating system.

It is the owner of ALL system processes, directories, files and devices.

It have full supreme access to do everything anytime.

Any user have access restricted to own process, directories and files.

TRY DON'T BE THE ROOT USER, USE SUDO ONLY WHEN NEED.

SUID and SGID

Allows an executable file to run with the file owner's privileges.

ALWAYS VERIFY

Find SUIDs: `find / -perm /4000`

Find SGIDs: `find / -perm /2000`

Find both: `find / -perm /6000`

ALSO

Find nouser nogroup: `find / -nouser -nogroup`

Find user files: `find / -user uid`

keep clean

do at end of an install

```
find / -perm -4000 -o -perm -2000 -type f 2>/dev/null | sort > suids
```

do periodically

```
find / -perm -4000 -o -perm -2000 -type f 2>/dev/null | sort > tests
```

verify

```
diff suids tests
```

verify which package

```
dpkg -S /path/to/binary
```



Keyboard controls, control^ alt~

^q start terminal output XON

^s stop terminal output XOFF

^c interrupt process

^d ends process

^z pause process

^p previous line command

^n next line command

^r search line command history

^l clear terminal (is a L not a I)

^a go to the start of line.

^e go to the end of line.

^u cut to start of the line.

^k cut to end of the line.

^w cut the word before

^y paste cut text

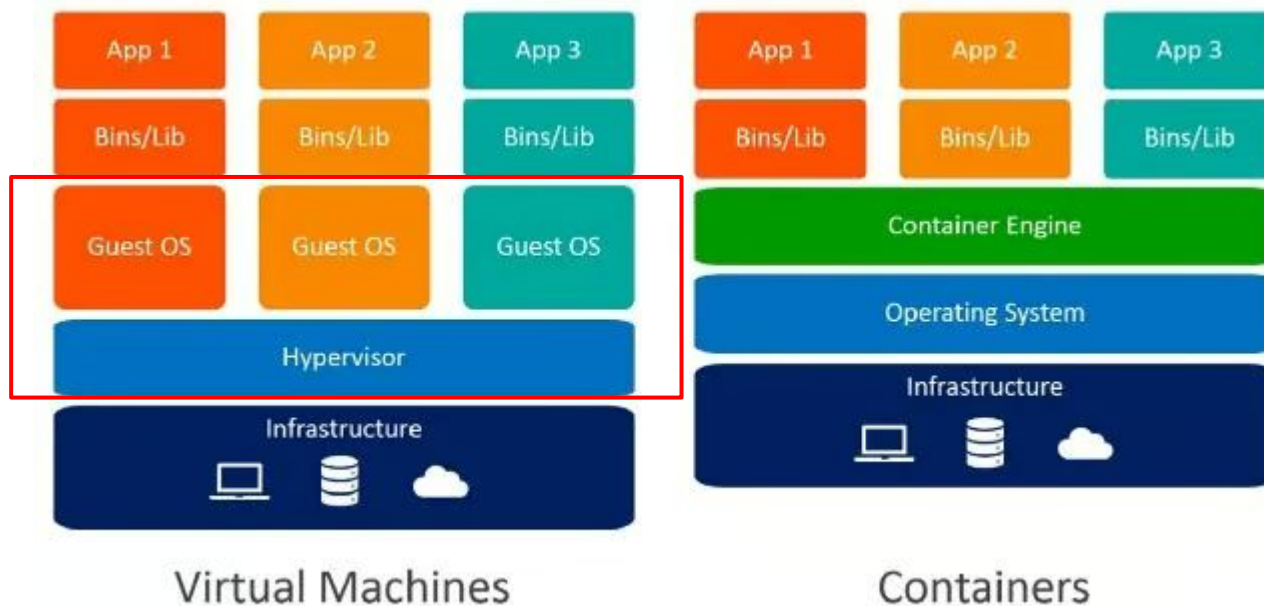
~d cut the word after

~b move a word backward

~f move a word forward



Virtual Machines or Containers



Container is not a Virtual Machine

the Container is a platform that utilizes native Linux kernel features like **namespaces** and **cgroups** to isolate processes. It shares the host system's kernel, making it lightweight and fast.

the **KVM** is a full virtualization technology that turns the Linux kernel into a hypervisor to run separate Virtual Machines (VMs) with their own dedicated guest operating systems.

A container could run a KVM by QEMU :)

```
docker run --device=/dev/kvm -it ubuntu bash
```

```
docker run --device=/dev/kvm -it windows11 sqlserver
```

thanks to <https://github.com/dockur/windows>

Container

Isolation guarantee

Resource limitation

Environment definition

Restricted namespaces

There is NO hypervisor

LXC (kernel tweaks)

cgroups (hardware access)

capabilities (root privileges)

namespaces (all but above)

<https://www.youtube.com/watch?v=x1npPrzyKfs>

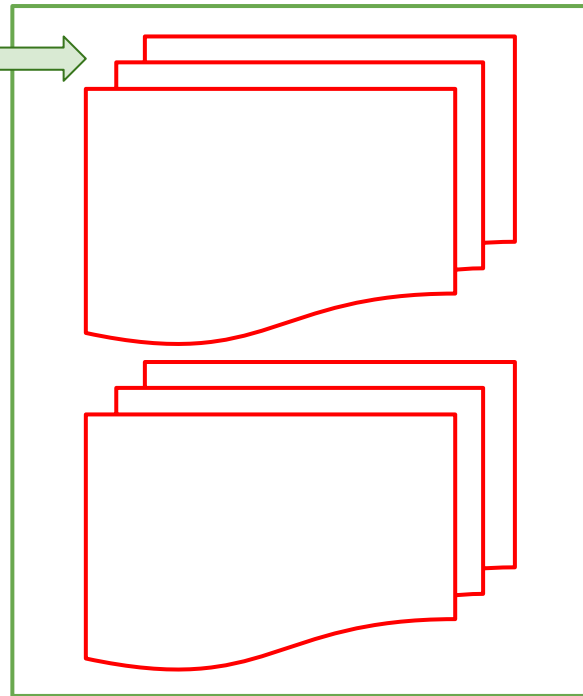
Kernel

Filesystems
Libraries
Devices
Sockets
Pipes
Namespaces
Processes



Containerd

Volumes
Libraries
Devices
Sockets
Pipes
Namespaces
Processes



containerd, users

Username do not matter, only the UID and GID do.

All user limits (*ulimits*) are inherited from the host.

Inside the container the UIDs and GIDs are “namespace exclusive”.

Files created by the container's root user belong to the host's root user.

→ Try Never Use Root User ←

In `/etc/docker/daemon.json`, use: `{ "userns-remap": "default" }`

must create in container `/etc/{subuid,subgid}` with eg: `default:200000:65536`

In command line, use: `docker run -u myuser:myuser ...`

containerd, volumes

A disk partition formatted with a file system could be mounted to be used as a part of directory tree.

Any local or remote directory tree mounted into the image's directory tree is a volume.

It inherits the host's metadata for owner, group and permissions.

A volume hides the previous content of the mount point.

named volumes, at `/var/lib/docker/volumes/`, as `this_volume:/pool`

bind volumes, at any host directory, as `/path/to/wherever:/pool`

containerd, networks

A container could share a network by using a virtual bridges, that works as switch (layer 2) for virtual ethernet pairs (veth)

That permits multiple networks inside same host, keep the isolation of networks and allow any network communicate with another network.

The services inside a container uses ports that could be mapped into host public ports to allow external access.

That permits a host have multiple internal servers at same port with only one exposed.

eg apache at 10.1.1.100:80 and nginx at 10.1.1.102:80 etc



containerd, images

An image is a directory tree containing selected files.

It is a file in tar format and compressed with gzip or zstd.

It also contains all necessary metadata about volumes, resources, capabilities, cgroups and namespaces.

The content of an image is immutable.

A registry is reliable repository where can pull or push images.

All images have a name (and tags) and a hash (SHA-256).

containerd, containers

A container is an image unpacked and expanded onto an overlay filesystem, with mounted volumes and running processes.

It can be controlled from the host to execute commands and share files or directories.

A container is mutable and can be saved as a new image.

Containers have an user defined name or a docker will choose a default name as a random combination of an adjective and a scientist's or hacker's last name.

windows images and containers

Microsoft provides windows images as

`://microsoft.com` or `mcr.microsoft.com/dotnet/runtime`

OR use

`github.com/dockur/windows`

OR use

`linux wine`

only for internet servers, can not use graphics and can not be interactive,



containerd: overlayfs

the overlayfs is a union filesystem with copy-on-write (CoW).

it works with 3 directories: base, diff and work;

- base is the lower directory, remains unchanged;

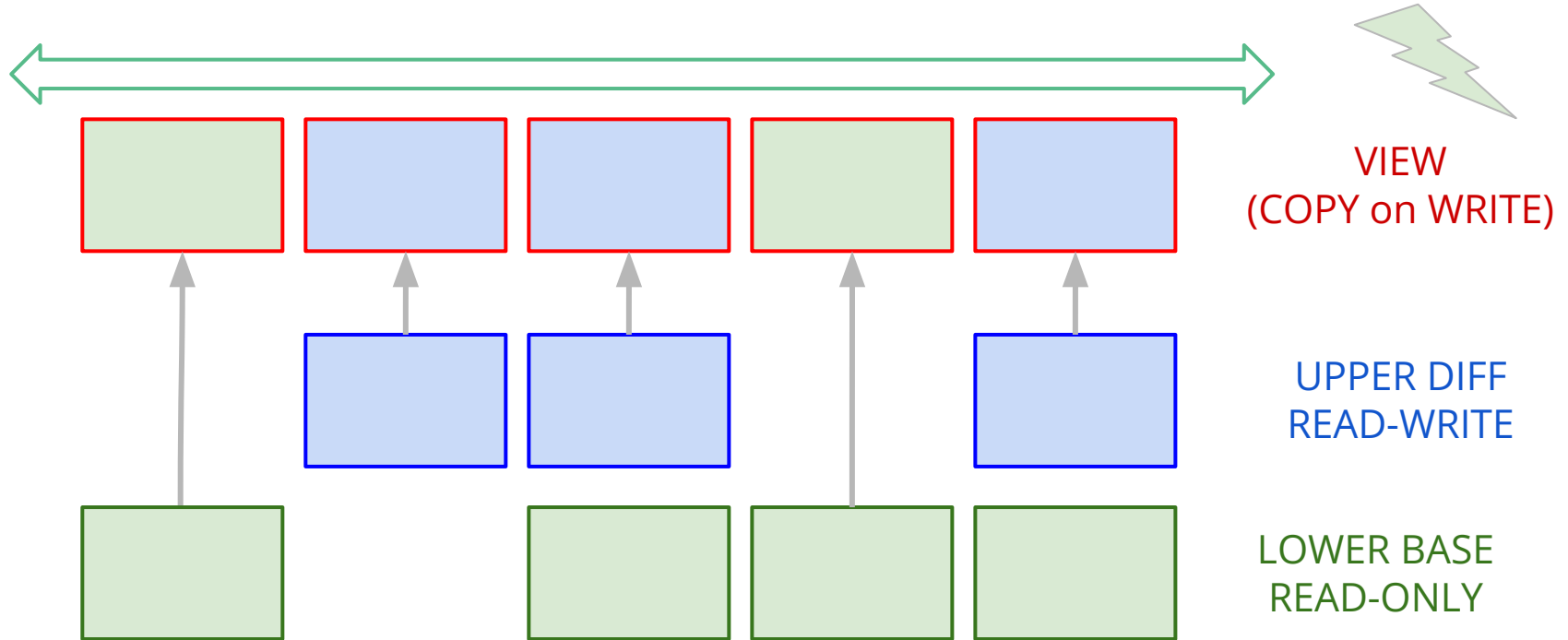
- diff is the upper directory, keep the differences;

- work is the work directory, used as temporary;

the result is merged in a single directory view.

<https://www.terriblecode.com/blog/how-docker-images-work-union-file-systems-for-dummies/>

overlayfs CoW, a sequence of blocks





Docker, Kubernetes, Podman, etc

Docker, Podman, Kubernetes are container management systems.

Dockerfile → Image → Container

Cluster → Controller node + Worker node → Swarms or Pods

Swarms and Pods are groups of containers that share the same network, storage, and lifecycle.

Docker and Kubernetes uses a daemon to control processes inside the container.

Podman does not use a daemon.

docker and kubernetes

Docker Swarm are best for small teams needing quick setup; Kubernetes Pods are best for complex and massive-scale applications;

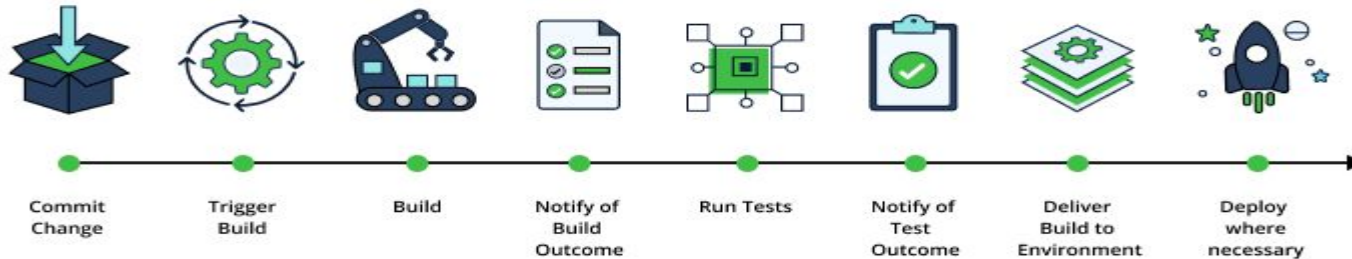
	dockers	kubernetes
Complexity	low	high
Scalability	limited swarms (~ 1024 nodes)	massive pods (thousands nodes)
Primary Role	Container packaging	Container orchestration

Container, registry

A registry is a storage system for container images at a reliable server.

Must keep a supply chain of security for development lifecycle to ensure the integrity of code and third-party dependencies.

CI/CD Pipeline



Image, manifest.json

```
"identification": "uuid:140dc68a-7e1c-11f1-83ca-00155d947c6a",  
"mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",  
"digest":  
"sha256:9834876dcfb05cb167a5c24953eba58c4ac89b1adf57f28f2f9d09af107ee  
8f0", "size": 32654 ,  
"annotations": {  
  "org.opencontainers.image.title": "minimal",  
  "org.opencontainers.image.version": "v1.0.0",  
  "org.opencontainers.image.ref.name": "minimal:latest",  
  "org.opencontainers.image.source": "https://github.com/",  
  "org.opencontainers.image.revision": "e7d8c9b"  
},
```

and so on and so forth



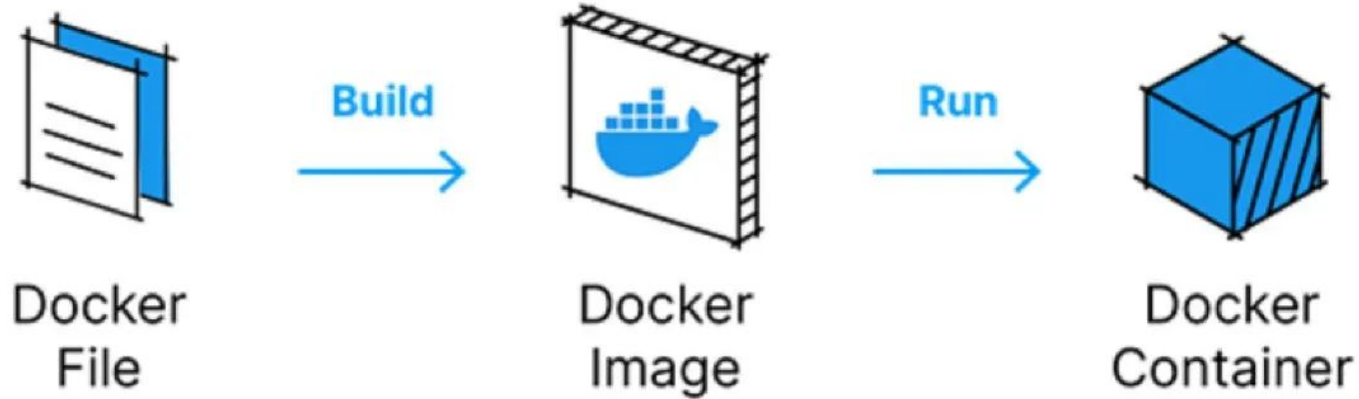
Dockerfile

A Dockerfile is a standard template for creating a container image.

It is structured as a blueprint, with sequence of instructions to make directories, variables, definitions and processes.

The docker-compose is used to execute a Dockerfile.

A Dockerfile is always executed inside the container.



Dockerfile, part 1

Use an official, base image

FROM debian:bookworm-slim AS builder

Set the working directory in container, used as myuser

WORKDIR /app

Create user and group, vary UID and GID conform applications

RUN adduser -uid 1000 -gid 1000 myuser && chown -R myuser:myuser /app

Copy host files to container directory

COPY --chown=myuser:myuser --chmod=755 build.sh dist.sh install.sh .

Copy host files to container directory, the ignore file

COPY .dockerignore /

Dockerfile, part 2

Update the container

RUN apt update && apt upgrade -y

Install necessary packages

RUN apt install curl git build-essential autoconf automake ca-certificates gnupg

Execute the scripts

RUN build.sh

Execute the scripts

RUN dist.sh

uninstall necessary packages

RUN apt purge curl git build-essential autoconf automake ca-certificates gnupg

Dockerfile, part 3

Use an official, base image

FROM debian:bookworm-slim AS worker

Copy necessary files

COPY --link --from=builder /app/dist /usr/local/

Execute the script

RUN install.sh

Clean up

RUN apt autoremove -y && apt-get clean -y && rm -rf /var/lib/apt/lists/*

Dockerfile, part 4

```
# Define environment variables  
ENV MAJOR=1. MINOR=02. PATCH=02  
ENV TIMEOUT=30 LOG_LEVEL=info DEBUG=info
```

```
# Switch to non-root system user  
USER myuser
```

```
RUN chown -R myuser:myuser /app
```

```
# App listens on  
EXPOSE 5000
```

```
ENV PORT=5000
```

Dockerfile, part 5

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \\  
  CMD wget -qO- <http://localhost:3000/health> || exit 1
```

```
# Run the application when the container starts  
CMD ["bash", "apps.sh"] # ENTRYPOINT ["/myapp"]
```


scan for vulnerabilities

Linux

```
curl -sfL sudo \
```

```
https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/install.sh |  
sudo sh -s -- -b /usr/local/bin
```

Scan an image

```
trivy image myapp:latest
```

Output as JSON for parsing

```
trivy image --format json --output results.json myapp:latest
```

Scan the image filesystem (for more detail)

```
trivy image --scanners vuln,secret,misconfig myapp:latest
```



Sizes matters

Base Approx. node.js, size:

debian, node:22 ~1.1GB

debian, node:22-slim ~220MB

alpine, node:22-alpine ~155MB

debian, gcr.io/distroless/nodejs22-debian12 ~145–177MB

<https://bhavyansh001.medium.com/6-dockerfile-habits-that-quietly-wreck-small-projects-e7cfd7a11f64>

.dockerignore

```
cat << EOF > .dockerignore
```

```
.git/  
.gpg/  
.ssh/  
.env/  
*.env  
*.log  
*.err  
*.cry  
*.swp  
*~  
*.o  
*.a  
EOF
```

```
cat << EOF >> .dockerignore
```

```
*.env.*  
*.pem  
*.key  
*.p12  
.secrets  
.secrets/*  
credentials.json  
*-account*.json  
EOF
```

defaults /etc/docker/daemon.json

```
{  
  "userns-remap": "default", "data-root": "/mnt/docker-data" ,  
  "live-restore": true,  
  "log-driver": "json-file", "log-opts": { "max-size": "10m", "max-file": "3" },  
  "userland-proxy": false, "icc": false,  
  "no-new-privileges": true, "dns": ["8.8.8.8", "1.1.1.1"],  
  "metrics-addr": "127.0.0.1:9323", "experimental": false  
}
```

Docker Swarm

Container

A single isolated runtime instance of an image.

Service

A declarative blueprint for a group of replicas from identical containers (called tasks) that share dependencies.

Stack

A collection of interrelated services that share dependencies and are managed together as a single application.

Docker Swarm

service # service is one application with replicas

ls lists service names, modes, replicas, images, and ports

create creates a swarm service

scale define the scale of replicas a service

update update running service

logs retrieve the logs from a service

stack # stack is many application with replicas

deploy starts a stack of services in a compose file

service deploy

```
# containers as a Swarm Service
```

```
service:
  web:
    deploy:
      mode: replicated
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first
```

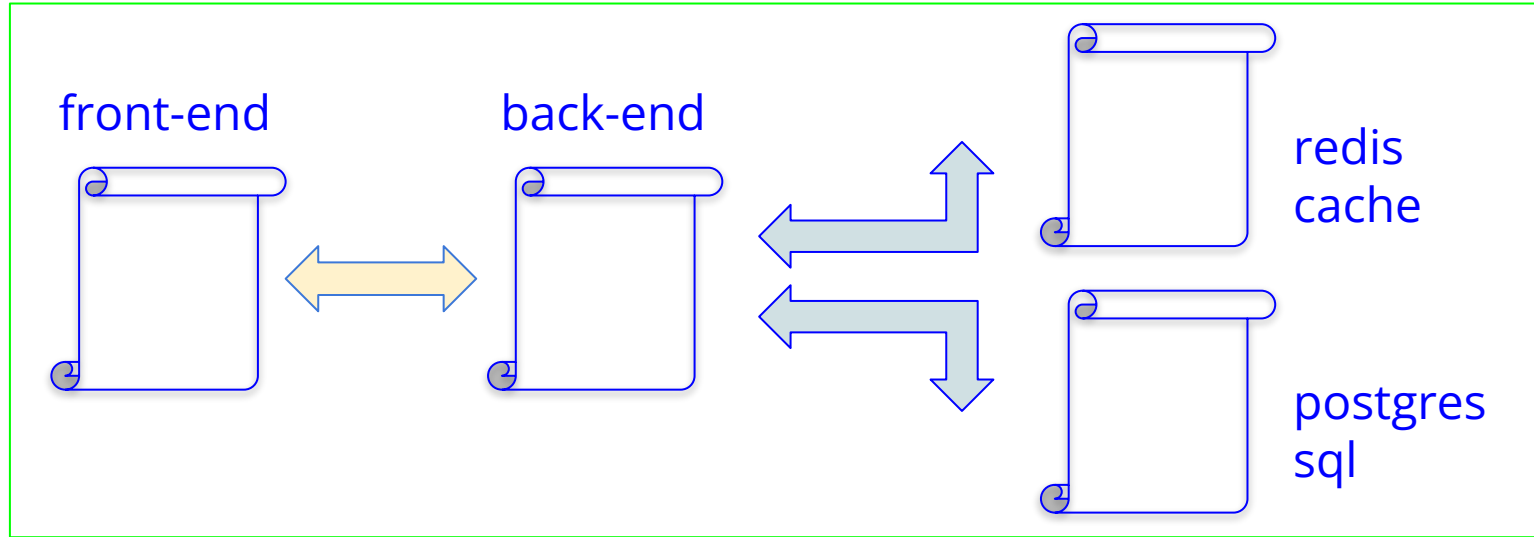
```
# containers as a Swarm Service
```

```
restart: unless-stopped
restart_policy:
  condition: on-failure
  max_attempts: 3
  window: 120s
resources:
  limits:
    cpus: '0.50'
    memory: 512M
```




containerd, cluster or stack

A cluster is a set of containers that shares networks, volumes and services. It is called swarm by docker and pods by kubernetes.



compose.yaml, part 1

services:

```
# --- FRONTEND SERVICE (Pre-built Image) ---
```

```
frontend:
```

```
  image: nginx:alpine
```

```
  user: "1000:1000"
```

```
  container_name: app_frontend
```

```
  ports:
```

```
    - "80:80"
```

```
  networks:
```

```
    - frontend_net
```

```
  restart: unless-stopped
```

compose.yaml, part 2.1

--- BACKEND SERVICE (Custom Built Image via Dockerfile) ---

backend:

build: ./backend # directory with Dockerfile

user: "1000:1000"

container_name: app_backend

ports:

- "5000:5000"

environment:

- REDIS_HOST=cache

- DB_HOST=database # use docker internal DNS resolution

networks:

- frontend_net # Can talk to frontend

- backend_net # Can talk to DB and Cache

compose.yaml, part 2.2

```
# --- BACKEND SERVICE (Custom Built Image via Dockerfile) ---  
  depends_on:  
    database: condition: service_healthy # wait DB health check  
    cache:    condition: service_started  
  restart: unless-stopped
```

compose.yaml, part 3

```
# --- CACHE SERVICE ---
```

```
  cache:
```

```
    image: redis:7-alpine
```

```
    user: "1000:1000"
```

```
    container_name: app_cache
```

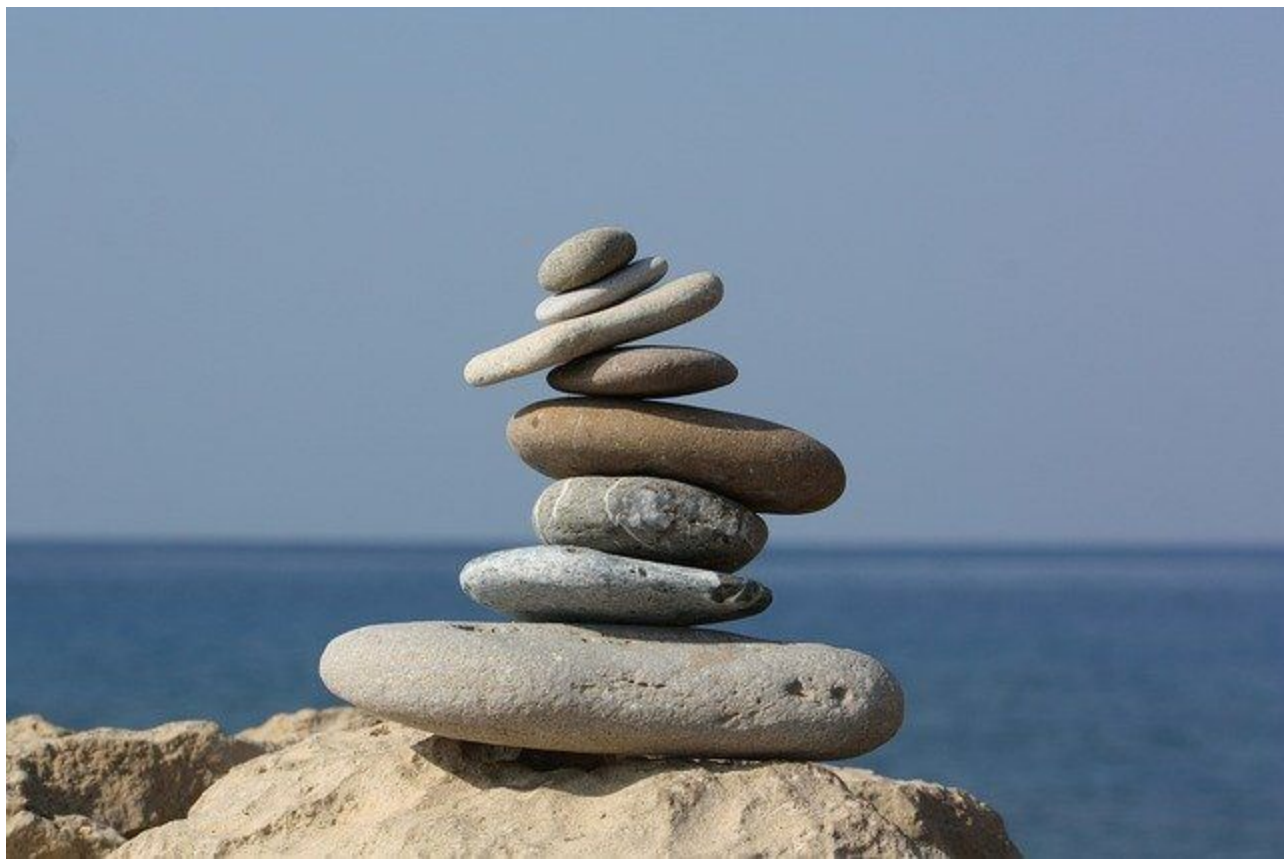
```
    volumes:
```

```
      - cache_data:/data
```

```
    networks:
```

```
      - backend_net      # Isolated from the frontend service
```

```
    restart: unless-stopped
```



compose.yaml, part 4.1

```
# --- DATABASE SERVICE (Persistent Volume) ---
```

```
  database:
```

```
    image: postgres:16-alpine
```

```
    user: "1000:1000"
```

```
    container_name: app_database
```

```
    environment:
```

```
      POSTGRES_USER: ${DB_USER:-admin}
```

```
      POSTGRES_PASSWORD: ${DB_PASSWORD:-secretpass}
```

```
      POSTGRES_DB: app_db
```


compose.yaml, part 4.2

```
# --- DATABASE SERVICE (Persistent Volume) ---  
# Persists database records on the host  
volumes:  
  - db_data:/var/lib/postgresql/data  
networks:  
  - backend_net      # Isolated from the frontend service  
healthcheck:  
  test:  
    ["CMD-SHELL", "pg_isready -U ${DB_USER:-admin} -d app_db"]  
  interval: 10s  
  timeout: 5s  
  retries: 5  
restart: unless-stopped
```

compose.yaml, part 5

--- NETWORKS SPECIFICATION ---

networks:

```
    frontend_net:      # External network for web traffic
                        driver: bridge
    backend_net:       # Internal network for secure service
                        driver: bridge
```

--- VOLUMES SPECIFICATION ---

volumes:

```
    db_data:          # named volume managed by Docker for database storage
    cache_data:       # named volume managed by Docker for redis storage
```



Docker command

login	Authenticate to a registry
pull	Download an image from a registry
push	Upload an image to a registry
search	Search register for images
logout	Log out from a registry
version	Show the Docker version information
info	Display system-wide information

Docker command

help	Display this help
run	Create and run a new container from an image
exec	Execute a command in a running container
ps	List containers
image	List images
build	Build an image from a Dockerfile
service	manage information about a docker swarm

Docker command

system	manages system
images	manage images
containers	manage containers
networks	manage networks
volumes	manage volumes
history	List the all history of a image
inspect	report all information of a image
secret	create a secure file or directory mounted in /run/

Docker command container_name

start	start the container
stop	stop the container
restart	restart the container
pause	pause the container
unpause	unpause the container
logs	retrieve the logs from container
inspect	report all information about container
stats	retrieve the statistics from container
diff	report modifications on filesystems of container

as bash functions at ~/.bashrc

```
dsh() { docker exec -it "$1" bash 2>/dev/null || docker exec -it "$1" sh }
```

```
dlog() { docker logs -f --tail 50 -t "$1" }
```

```
denv() { docker inspect --format '{{range .Config.Env}}{{println . }}{{end}}' "$1" |  
sort }
```

```
dmnt() { docker inspect --format '{{range .Mounts}}{{println .Source "->"  
.Destination}}{{end}}' "$1" }
```

```
dnet() { docker inspect --format '{{range $name, $net := \  
.NetworkSettings.Networks}}{{println $name ":" $net.IPAddress}}{{end}}' "$1" }
```

```
dsts() { docker stats format 'table \  
{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}\t{{.MemPerc}}\t{{.NetIO}}\t{{.BlockIO}}' }
```




Poster service

```
$ docker run hello-world
```

```
$ docker pull gitlab/gitlab-ce:latest
```

```
$ docker run --detach \  
  --hostname gitlab.localhost --name gitlab.public --restart always \  
  --publish 4430:443 --publish 8000:80 --publish 2200:22 \  
  --volume /srv/gitlab/config:/etc/gitlab \  
  --volume /srv/gitlab/logs:/var/log/gitlab \  
  --volume /srv/gitlab/data:/var/opt/gitlab \  
  gitlab/gitlab-ce:latest
```

```
$ docker exec -it gitlab.public bash
```

Poster process

`sudo systemctl status docker`

`sudo systemctl status`

The gitlab-ce image is a complete example of ideal internet system

You can't fix something unless
you know what's broken.

Trinity Tran, TED Talks

Distros

Debian ****slim**** does contain almost all need for common use;

Alpine uses musl instead of glibc, could break packages;

Distroless images does not contain any shell;

<https://github.com/googlecontainerertools/distroless>

distroless:base contains libc and libssl

distroless:static do not contains libc or any shared lib

Host, what limits it?

the bogomips value should be close to twice the clock frequency (GHz)

```
cat /proc/cpuinfo | \
```

```
    grep -E 'processor|model name|bogomips|cpu cores' | tail -4
```

cache impacts hyper thread for code (L1c) and data (L1d) sizes swaps

```
cat /sys/devices/system/cpu/cpu0/cache/index*/size
```

memory Total, Swap and Huge direct impacts performance

```
cat /proc/meminfo | grep -E 'Total|Swap|Huge'
```

Host, where am I ?

```
uname -a  
lsblk  
free -h  
mount -l  
ip addr  
ip route  
systemctl list-units --type=service --state=running  
df -h  
crontab -l
```

<https://systemweakness.com/11-linux-commands-i-run-on-every-fresh-installation-344269910dd6>

Host, connections

Check the maximum allowed entries

```
sysctl net.netfilter.nf_conntrack_max
```

Check current conntrack usage

```
sysctl net.netfilter.nf_conntrack_count
```

Watch conntrack usage in real time

```
watch -n1 'cat /proc/sys/net/netfilter/nf_conntrack_count'
```

Host, connections

list actual value

```
sudo sysctl -a | grep -i net.netfilter
```

temporarily change

```
sudo sysctl net.netfilter.nf_conntrack_max=262144
```

persistent change

```
echo "net.netfilter.nf_conntrack_max=262144" >> /etc/sysctl.conf
```

restart system to apply the configuration

```
sudo sysctl --system
```

Host, sysctl

temporary changes

sysctl <tunable class>.<tunable>=<value>

permanent changes, order matters

echo "<tunable class>.<tunable>=<value>" >> /etc/sysctl.conf

must restart

sudo sysctl --system

or

service procps force-reload

Host, systemd

/etc/systemd/docker.service

MemoryMax=2G

MemoryHigh=1.5G

/etc/systemd/journald.conf, check with: journalctl --disk-usage

SystemMaxUse=500M

SystemKeepFree=1G

Host, systemd

```
## /etc/systemd/<service>.service
```

```
Environment="PATH=/usr/local/bin:/usr/bin:/bin"
```

```
## verify
```

```
cat /proc/$(pgrep <service>)/limits | grep "open files"
```



Dockerfile, use cache

docker cache is always a directory

```
RUN --mount type=cache,target=/var/cache/apt \  
    --mount type=cache,target=/var/lib/apt/lists \  
    apt-get update && apt-get install -y curl
```

```
RUN --mount type=cache,target=/usr/local/ \  
    cd build && make && make test && make install && \  
    cp -var /usr/local dist
```

Dockerfile, use secret

docker secret is always a file

```
RUN --mount type=secret,id=mysecret,target=/root/.mysecret \  
    <wherever needs some secret text>
```

```
RUN --mount type=secret,id=api_key \  
    export OPENAI_API_KEY=$(cat /run/secrets/api_key) && \  
    ./setup.sh
```

```
docker build --secret id=api_key,src=.env .
```


Compose, part 6

mounted at /run/secrets in RAM,

must include */secrets/ in .gitignore

secrets:

db_password:

file: ./secrets/db_password.txt

Compose, part 7

services:

api:

init: true

build: .

develop:

watch:

- action: sync

path: ./src

target: /app/src

- action: rebuild

path: package.json

<https://liu-111.medium.com/15-docker-techniques-that-cut-build-times-and-image-sizes-5904dcf59354>

Dockerfile, part A

```
# syntax=docker/dockerfile:1
```

```
FROM golang:1.26 AS build
```

```
WORKDIR /src
```

```
# Dependency layer: only rebuilds when go.mod/go.sum change
```

```
COPY go.mod go.sum ./
```

```
RUN --mount=type=cache,target=/go/pkg/mod go mod download
```

Dockerfile, part B

Source layer: rebuilds on code changes, but the caches above persist

COPY ..

RUN --mount=type=cache,target=/go/pkg/mod \

--mount=type=cache,target=/root/.cache/go-build \

CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" \

-o /bin/app ./cmd/server

Dockerfile, part C

Runtime: ~2 MiB base, includes CA certs and tzdata, runs as non-root

FROM gcr.io/distroless/static-debian13:nonroot

COPY --from=build /bin/app /app

EXPOSE 8080

ENTRYPOINT ["/usr/bin/bash", "/app/doit.sh "]

<https://medium.com/@danielvalev/your-docker-image-is-1-2-gb-because-you-stopped-reading-tutorials-in-2019-e31b314b14de>

More JSONs

JSON5 allows comments and trailing commas for more configuration files.

JSONC (JSON with comments, used by VSCode) does something similar.

BSON (Binary JSON) powers MongoDB's internals.

EJSON (Extended JSON) adds additional type support.

GeoJSON structures geographic data.

And there are probably a dozen others I'm missing.

<https://levelup.gitconnected.com/jsonl-the-format-we-didnt-know-we-needed-405e3ed407ab>

SSH easy

```
# create a cert
ssh-keygen -t ed25519

# copy the id to all hosts
ssh-copy-id <user@host.domain>

# add to configuration
cat << EOT >> ~/.ssh/config
Host webserver
    User myuser
    Hostname xx.xx.xx.xx
EOT
```

```
# connect to a host

ssh webserver

ssh admin@192.168.1.100 \
    "local_script.sh"

ssh admin@192.168.1.100 \
    < local_script.sh
```

Practice

Stages

1. Store all Dockerfile or Kubernetes manifests in a Git repository
2. Require pull requests for any change
3. Use a staging environment to test changes before production
4. Tag releases so you can roll back to a known good state

Segments

1. production // Live, customer-facing workloads
2. staging // Pre-release testing
3. develop // Active development builds
4. monitoring // Logs, metrics, observability tools
5. infrastruct // Shared infrastructure services

Old pipeline, common

1. [GitHub Push]
2. [Build Docker Image] ————— 6 min
3. [Push to Registry] ————— 2 min
4. [Pull on Server] ————— 3 min
5. [Run Container] ————— 30 sec

New pipeline, alternative

1. [GitHub Push]
2. [Run Tests] ————— 45 sec
3. [rsync to server] ————— 8 sec
4. [systemctl restart] ————— 2 sec

<https://medium.com/@ArkProtocol1/docker-is-dead-weight-i-rebuilt-our-pipeline-without-it-and-cut-costs-60-d45aceb0a591>

Paranoides

```
find / -perm /6000
```

```
find / -type d \( -name "*bin" -o -name "*sbin" \) -prune \  
    -o -type f -executable -print 2>/dev/null
```

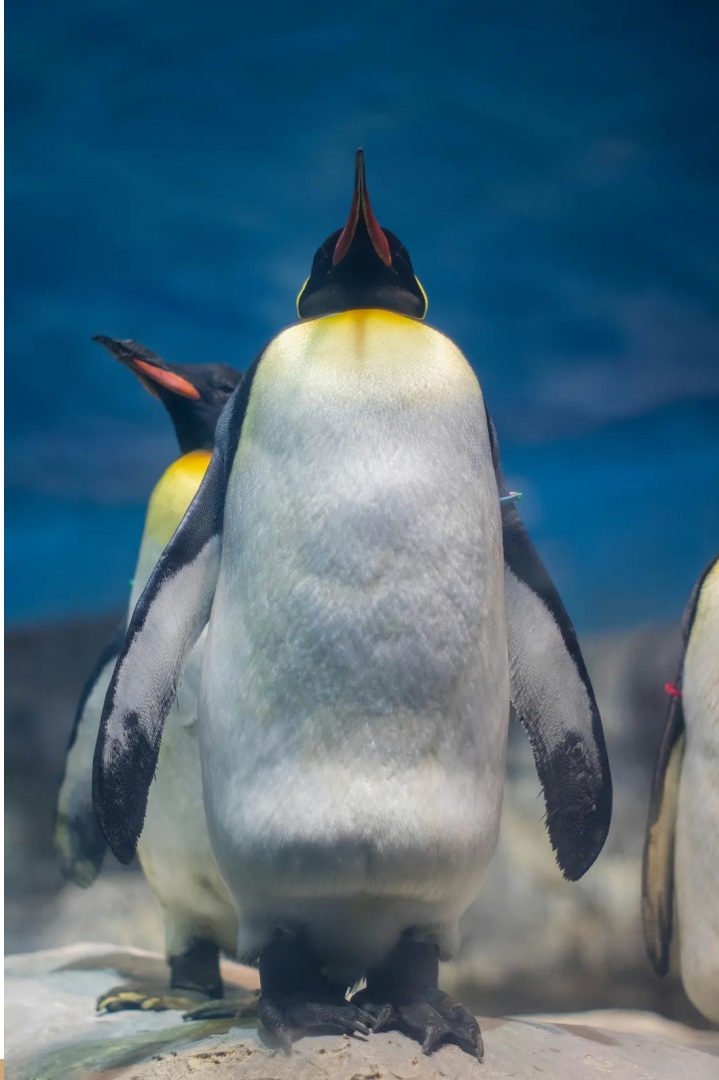
```
cp /usr/bin/chmod /root
```

```
/root/chmod 000 /bin/* /sbin/* /usr/bin/* /usr/sbin/* \  
    /usr/local/bin /usr/local/sbin /usr/libexec
```

```
/root/chmod 644 /bin/* /sbin/* /usr/bin/* /usr/sbin/* \  
    /usr/local/bin /usr/local/sbin /usr/libexec
```

how do the magics ?

uuid
unshare
lsns
ip
iptables
mount --bind
pivot_root
chroot
runc



references

<https://blog.nginx.org/blog/what-are-namespaces-cgroups-how-do-they-work>

<https://erzeghi.medium.com/yet-another-brief-history-of-container-d-2962eac9679e>

<https://opencontainers.org/>

<https://json-schema.org/>

<https://ms.xt3.it/teach/sysadm/ulsah.pdf>

<https://blog.lizzie.io/linux-containers-in-500-loc.html>

<https://medium.com/@pathirage/awk-in-linux-ae316bf6af89>

references

<https://www.youtube.com/watch?v=x1npPrzyKfs>

<https://dev.to/rowleks/data-serialization-a-concise-guide-to-json-yaml-toml-and-more-23co>

<https://blog.stackademic.com/practical-jq-for-developers-parse-json-from-the-terminal-d6caac870d4f>

<https://ercan.cloud/take-your-gitlab-backup-everyday-if-it-works-in-docker/>

<https://www.hostinger.com/br/tutoriais/docker-cheat-sheet/>

<https://medium.com/@sandunlakshan213/10-modern-cli-tools-you-should-try-in-2026-18ce4e16c28e>

references

<https://medium.com/@bektiaw/5-bash-tricks-that-make-you-faster-in-the-terminal-9c38af18e054>

<https://medium.com/@keshrianjani20/shell-aliases-every-devops-engineer-should-have-ea6a9e994029>

<https://contabo.com/blog/docker-commands-guide-for-developers/>

<https://medium.com/@atnofordevops/write-dockerfiles-like-a-senior-devops-engineer-047f0543ee0e>

<https://blog.s10n.dev/15-cli-tools-that-replaced-postman-docker-desktop-slack-and-other-bloated-electron-apps-for-me-2a092745863c>

References

<https://binadit.com/tutorials/configure-linux-kernel-parameters-with-sysctl-for-system-optimization>

https://medium.com/@_jaydeepkarale/how-to-handle-secrets-7c89a7dea723

references

<https://github.com/jesseduffield/lazydocker>

<https://github.com/derailed/k9s>

<https://github.com/wagoodman/dive>

<https://github.com/jesseduffield/lazygit>

<https://github.com/mrjackwills/oxker>

<https://github.com/mikefarah/yq>

<https://github.com/imsnif/bandwhich>

<https://github.com/printfn/fend>

1,000+ commands. You need 20.



20

pwd

ls

cd

mkdir

cp

mv

rm

cat

less

head

tail

grep

find

chmod

sudo

ps

kill

curl

ssh

man

📁 /usr/bin 1,925 programs

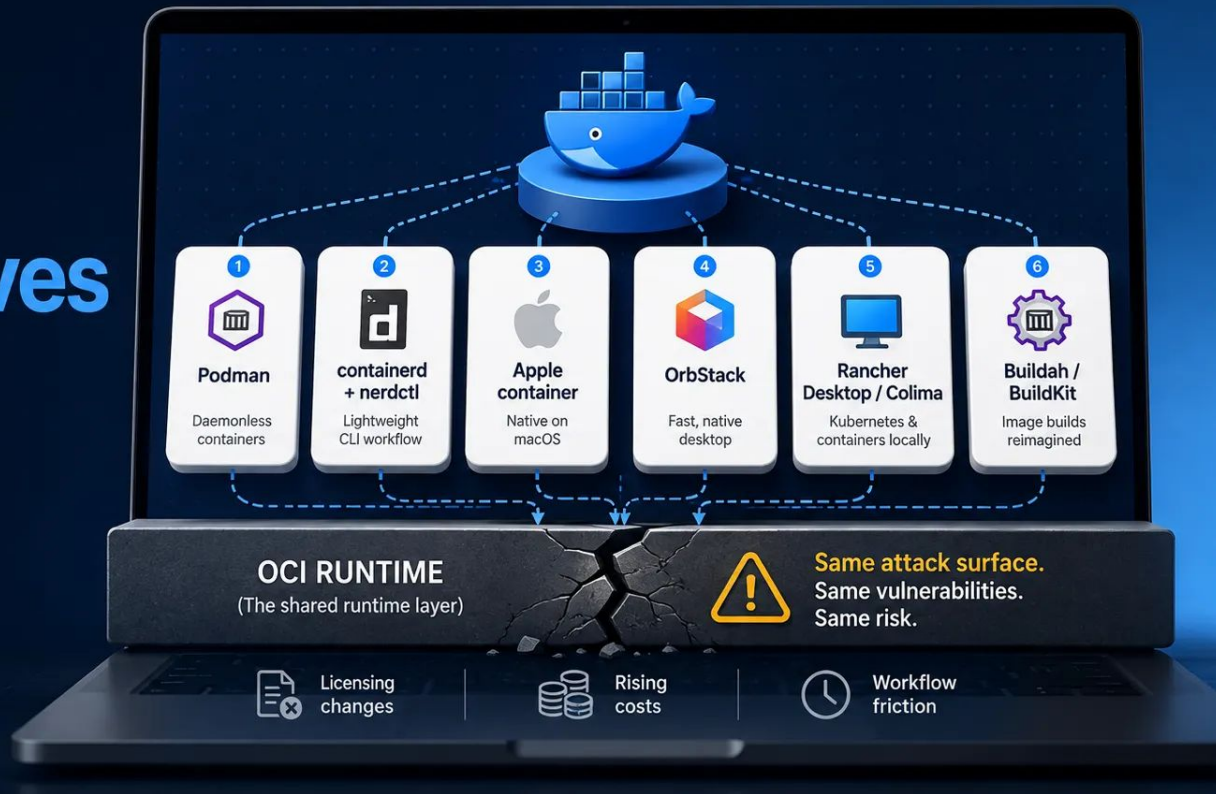
\$ _

Top 6 Docker Alternatives

...and the one thing
none of them fix

- \$ Licensing changes
- 📈 Rising costs
- 👥 Workflow friction

↔ Teams are switching.
But the core issue remains.



Securing the Software Supply Chain



<https://jfrog.com/learn/software-supply-chain/>

[GitHub Push]

[Build Docker Image] — 6
min

[Push to ECR] — 2
min

[Pull on Server] — 3 min

[Run Container] — 30
sec

that is all folks



YAML, anchor and reference

Define the anchor

```
default_version: &version "1.24"
```

Use the alias services

```
web: image: "nginx:*version" # Resolves to "nginx:1.24"
```

& works as tuple of a key and a value

* assign the parameter value to attribute key

YAML, template

1. Base configuration template (using a hidden key '.' so it's ignored)

```
.db-template: &db_base
```

```
  adapter: postgresql
```

```
  host: localhost
```

```
  username: admin
```

```
  password: secured
```

<<: merge multiple parameters

YAML, template

2. Reusing and overriding configurations

development: <<: *db_base # Injects all keys from db_base

 database: dev_db # Adds a new key

production: <<: *db_base # Injects all keys from db_base

 database: prod_db # Adds a new key

 host: prod-cluster # Overrides the default

honeypot: <<: *db_base # Injects all keys from db_base

 password: easiest # Override the default



Last Seen

